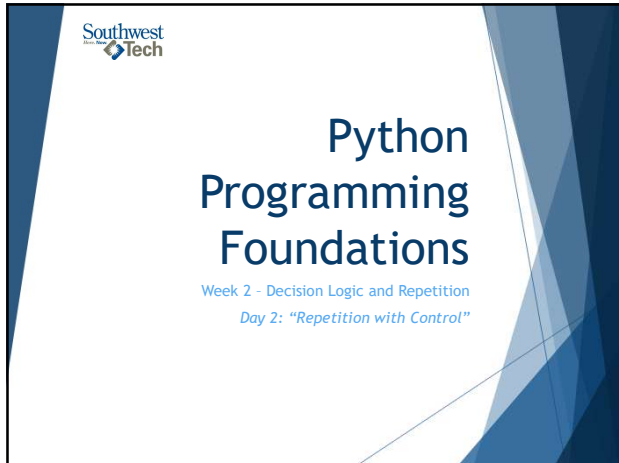




1



2



3



Logic Chooses, Loops Repeat

Decision logic chooses what happens.
Loop logic repeats something on purpose.

Week 2 now has two control-flow tools:

- branches for choosing
- loops for repeating

4



Two Ways to Control Flow

BRANCHES CHOOSE

Pick one path based on a condition.

```

graph TD
    A[Check Condition] -- True --> B[Do This Path A]
    A -- False --> C[Do That Path B]
    B --> D[Continue After the Choice]
    C --> D
    
```

Branches pick one path. Then the program continues.

LOOPS REPEAT

Repeat an action while a condition is True.

```

graph TD
    A[Check Condition] -- True --> B[Do the Action Repeat This]
    B --> A
    A -- False --> C[Stop Loop Continue After Loop]
    
```

Loops repeat the action. They stop when the condition is False.

Logic Chooses, Loops Repeat

5



What Counts as Success Today

Repeat action -> update or advance
-> check stopping point.

↓

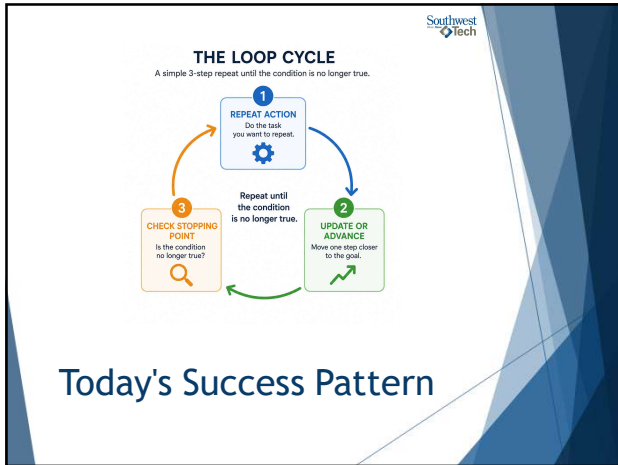
If you can explain those three parts,
you understand the loop.

↓

For every loop, ask:

What code repeats?	What becomes different?	How does the loop stop?
--------------------	-------------------------	-------------------------

6



7

What We Will Use Today

- ▶ `'for'`
- ▶ `'range()'`
- ▶ counters
- ▶ accumulators
- ▶ `'while'`
- ▶ stopping conditions

These tools let a program repeat work without duplicating the same lines.

8

Loop Concept Toolbox
Core ideas for building and understanding loops.

for Repeat a block of code a set number of times. Use when you know how many times to loop.	range() Generate a sequence of numbers for a loop. Examples: <code>range(0, 1, 2, 3, 4)</code> <code>range(1, 6)</code>	counter Keep track of how many times the loop has run. Start at 0 or 1. Increase each time through the loop.
accumulator Build up a total or result over multiple iterations. Clear at 0 for another value. Add to it each time through the loop.	while Repeat a block of code while a condition is true. Use when you do not know how many times to loop.	stopping condition The condition that ends the loop. The loop stops when this condition becomes false.

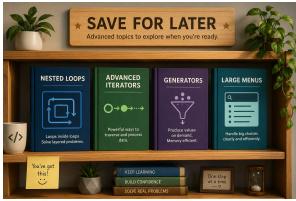
Choose the right tool for the job. Clear loops are reliable loops.

Today's Toolbox

9

Southwest
Tech

What We Will “Skip” For Now



- ▶ nested loops
- ▶ advanced iterators
- ▶ generator tools
- ▶ large menu programs

Instead: A good beginner loop is small enough to trace:

- ▶ first pass
- ▶ second pass
- ▶ last pass

10

Southwest
Tech

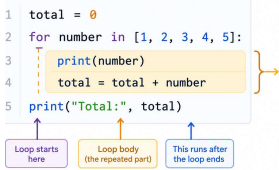
Ask What Repeats

Loop Example

```

1 total = 0
2 for number in [1, 2, 3, 4, 5]:
3     print(number)
4     total = total + number
5 print("Total:", total)

```



The repeated block is the work the loop performs each time. Before running a loop, identify the repeated action.

Examples:

- ▶ print a number
- ▶ add to a total
- ▶ ask for another value
- ▶ count down toward zero

11

Southwest
Tech

Ask What Changes

Loop in Action: State Changes
The same steps run each pass, but the values change.

PASS 1	PASS 2	PASS 3
counter = 1	counter = 2	counter = 3
if(counter): total = counter + 1	print(counter) counter = counter + 1	print(counter) counter = count
Output: 1	Output: 2	Output: 3

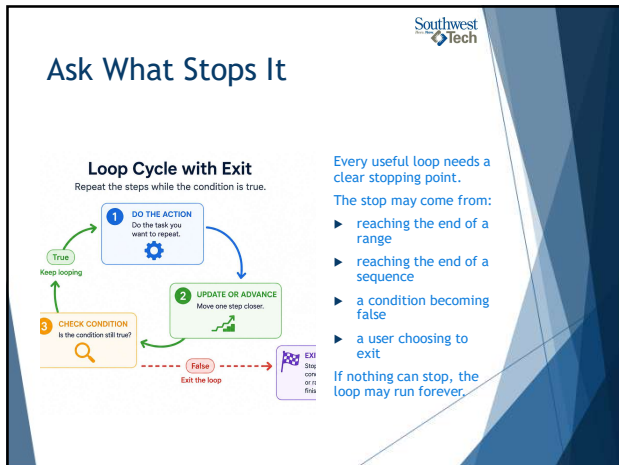
STATE CHANGES
The value of counter changes each pass (1 → 2 → 3 → ...). That is why the loop eventually stops.

A controlled loop usually has something that changes or advances.

That change may be:

- ▶ the next number from 'range()'
- ▶ a counter value
- ▶ a running total
- ▶ a user choice
- ▶ a condition becoming false

12



13

`for` Loops: Known Repetition

Southwest
Tech

A `for` loop is useful when the program knows what it is repeating over.

Common beginning pattern:

```
for number in range(5):
    print(number)
```

The range controls how many times the loop runs.

14



15



What to watch for

Watch the loop variable change each time the loop runs.

For each pass, identify:

- ▶ the current value
- ▶ the repeated output
- ▶ when the range is finished

16



range(5) in Action

```
for i in range(5):
    print(i)
```

Step 1: i = 0 → Step 2: i = 1 → Step 3: i = 2 → Step 4: i = 3 → Step 5: i = 4

range(5) produces: 0, 1, 2, 3, 4

Output (one line printed each time): 0 → 1 → 2 → 3 → 4

Demo 1: Counting

17



Accumulators Build A Result

An accumulator is a variable that collects or builds a value over repeated steps.

Common pattern:

```
total = 0
for number in range(1, 4):
    total = total + number
```

The total changes each time the loop runs.

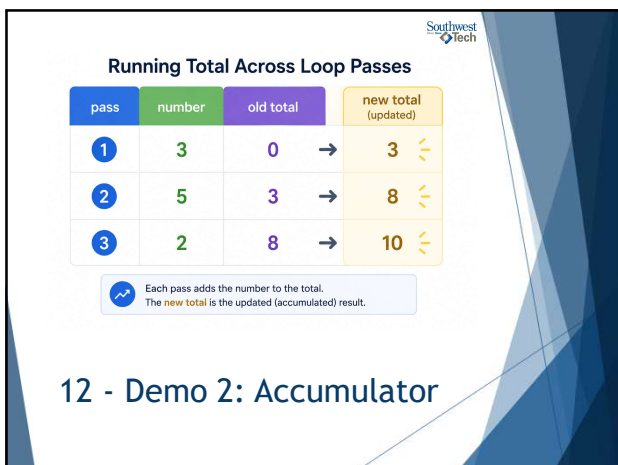
18



19



20



21

Southwest
Tech

`while` Loops: Condition- Controlled Repetition

A `while` loop repeats while a condition is `True`.

Basic shape:

```
while condition:  
    repeat_this()
```

The loop stops when the condition becomes `False`.

22

Southwest
Tech

Missing Updates Cause Trouble

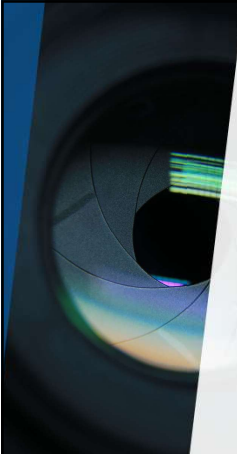


If the condition never changes,
the loop may never stop.

For a `while` loop, always ask:

- What condition starts the loop?
- What line changes the condition?
- What value makes the loop stop?

23



Demo #3

"While-Loop Countdown"

24



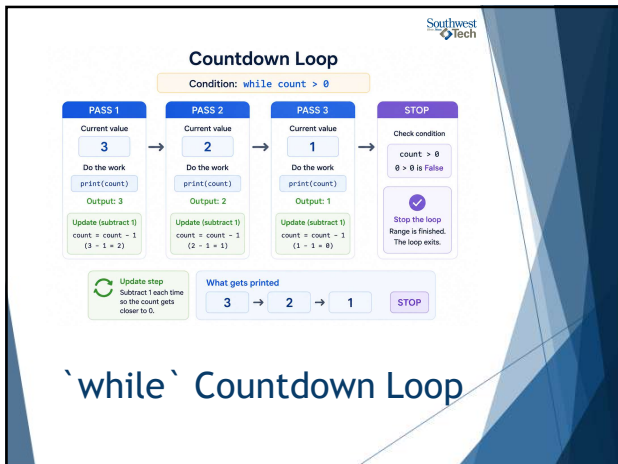
What to watch for

Watch the countdown change.

For each pass, identify:

- ▶ the current value
- ▶ the output
- ▶ the update
- ▶ the moment the loop stops

25



26



Optional Demo #4

"Input Loop w/Exit"

27



What to watch for

Repeated input is useful when the user may enter several values.

The program should still have a clear exit condition.

Watch for:

- ▶ the repeated prompt
- ▶ the user's choice
- ▶ the condition that ends the loop

28



From Demo to Lab

For Assignment 3, build one small program that repeats in a controlled way.

A good first version should include:

- ▶ one repeated action
- ▶ one clear stop point
- ▶ visible output
- ▶ no infinite loop

Before coding too much, write what repeats and what stops it.

29



Loop Assignment Options

Pick one to practice your loop skills.

1 Counting Program
Count from 1 to a number and print each value.

```

graph TD
    start(( )) --> repeat((repeat))
    repeat --> stop([stop])
            
```

Example:
Input: 5
Output: 1 2 3 4 5

Try different end numbers.

2 Total Accumulator
Add numbers and show the final total.

```

graph TD
    start(( )) --> repeat((repeat))
    repeat --> stop([stop])
            
```

Example:
Add: 4, 7, 2
Output: Total = 13

Use any 3 or more numbers.

3 Number Sequence
Generate a pattern or sequence.

```

graph TD
    start(( )) --> repeat((repeat))
    repeat --> stop([stop])
            
```

Example:
First 5 even numbers
Output: 2 4 6 8 10

Pick your own pattern.

4 Practice Loop
Create your own loop to solve a simple task.

```

graph TD
    start(( )) --> repeat((repeat))
    repeat --> stop([stop])
            
```

Example:
Up to you!
Make it useful and fun.

Be creative and test it!

★ Every loop has a **repeat** part and a **stop** point. Build it. Run it. Improve it.

Assignment 3 Options

30



Closing Success Check

If you can explain the loop, you control it.

Before submitting Assignment 3, test:

- ▶ what happens on the first pass
- ▶ what changes after one pass
- ▶ how the loop stops
- ▶ whether the output proves repetition happened

31



Closing Success Check

(Example)

Traceability Checklist

Follow the path from value to output.

1	Starting Value Set the initial value. Example: count = 1	☒
2	Condition Check the condition. Example: count <= 5?	☒
3	Branch Choose the branch. Example: Yes (true)	☒
4	Repeat Do the loop body. Example: Add count	☒
5	Change Update the value. Example: count = count + 1	☒
6	Stop Condition becomes false. Example: count > 5	☒
7	Output Show the final result. Example: Total = 15	☒

A clear path. Every time. Understand. Test. Trust.

32




Thank you!

Have Fun!

33
